LỜI MỞ ĐẦU

Trong bối cảnh công nghệ thông tin ngày càng phát triển mạnh mẽ, việc ứng dụng trí tuệ nhân tạo vào các hệ thống phần mềm đã trở thành một xu hướng tất yếu. Đặc biệt, với sự bùng nổ của dữ liệu phi cấu trúc như văn bản, tài liệu số, việc khai thác, xử lý và tóm tắt thông tin một cách nhanh chóng và hiệu quả đang là một bài toán quan trọng trong nhiều lĩnh vực.

Hiện nay, người dùng thường phải tiếp cận với một lượng lớn thông tin từ nhiều nguồn khác nhau, gây ra khó khăn trong việc nắm bắt nội dung cốt lõi. Do đó, việc xây dựng một hệ thống hỗ trợ tóm tắt văn bản tự động không chỉ giúp tiết kiệm thời gian mà còn nâng cao hiệu quả tiếp nhận thông tin.

Xuất phát từ nhu cầu thực tiễn đó, nhóm đã lựa chọn đề tài: **"***Xây dựng hệ thống tóm tắt văn bản tự động sử dụng mô hình AI kết hợp kỹ thuật RAG (Retrieval-Augmented Generation)***"**. Hệ thống được thiết kế nhằm cho phép người dùng nhập văn bản hoặc tải lên tài liệu, từ đó hệ thống sẽ xử lý và sinh ra nội dung tóm tắt một cách chính xác và dễ hiểu.

Trong quá trình thực hiện, nhóm đã áp dụng các kiến thức về phân tích và thiết kế hệ thống thông tin, kết hợp với các công nghệ hiện đại như FastAPI, cơ sở dữ liệu quan hệ, vector database và các mô hình học sâu để xây dựng một hệ thống hoàn chỉnh, có khả năng mở rộng và đáp ứng tốt yêu cầu thực tế.

Báo cáo này được chia thành các chương chính như sau:

- *Chương 1:* Giới thiệu tổng quan về đề tài, mục tiêu và phạm vi nghiên cứu
- *Chương 2:* Thiết kế và xây dựng hệ thống
- *Chương 3:* Thực nghiệm và đánh giá kết quả

Mặc dù đã có nhiều cố gắng trong quá trình thực hiện, tuy nhiên do hạn chế về thời gian và kinh nghiệm, báo cáo không tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý từ giảng viên hướng dẫn Lê Hoàng Sơn để có thể hoàn thiện hơn trong tương lai.

Nhóm chúng em xin chân thành cảm ơn thầy!

MỤC LỤC

Contents

CHƯƠNG 1: GIỚI THIỆU VỀ ĐỀ TÀI 5

1. Giải pháp công nghệ 5

2. Ý nghĩa thực tiễn 5

CHƯƠNG 2: THIẾT KẾ XÂY DỰNG 6

1. Xác nhận yêu cầu hệ thống 6

1.1. Sơ đồ mô hình hoạt động của hệ thống 6

1.2. Mô tả yêu cầu hệ thống 8

1.3. Biểu đồ Usecase 9

1.4. Biểu đồ tuần tự giao tiếp 9

2. Phân tích hệ thống 12

2.1. Mô tả phân tích hệ thống 12

2.2. Biểu đồ lớp (Class Diagram) 19

2.3 Biểu đồ trạng thái (State Diagram) 32

3. Thiết kế hệ thống 33

3.1 Thiết kế kiến trúc hệ thống 33

3.2 Lựa chọn công nghệ cho hệ thống 33

3.3 Thiết kế cơ sở dữ liệu 39

3.4 Thiết kế giao diện (Demo) 40

MỤC LỤC ẢNH

Ảnh: Luồng người dùng cơ bản 7
Ảnh: Usecase tổng quát 9
Ảnh: Biểu đồ tuần tự đăng ký/đăng nhập 9
Ảnh: Biểu đồ tuần tự tóm tắt văn bản 10
Ảnh: Biểu đồ tuần tự upload tài liệu 10
Ảnh: Biểu đồ tuần tự xem lịch sử tóm tắt 11
Ảnh: Biểu đồ tuần tự đánh giá 11
Ảnh: Kiến trúc Server (Backend) tổng thể 13
Ảnh: Luồng xử lý RAG 16
Ảnh: Minh họa giai đoạn Indexing 18
Ảnh: Biểu đồ lớp tổng quát 19
Ảnh: Biểu đồ trạng thái User 32
Ảnh: Biểu đồ trạng thái Document 32
Ảnh: Biểu đồ trạng thái Conversation 32
Ảnh: Biểu đồ trạng thái Message 32
Ảnh: Kiến trúc tổng thể của hệ thống 33
Ảnh: PostgreSQL 34
Ảnh: ChromaDB 35
Ảnh: Visual Studio Code 35
Ảnh: FastAPI 36
Ảnh: NextJS 37
Ảnh: Github 38

Ảnh: Docker 39
Ảnh: Biểu đồ ERD 39
Ảnh: Giao diện đăng nhập 40
Ảnh: Giao diện đăng ký 40
Ảnh: Giao diện chính 41
Ảnh: Giao diện hiển thị kết quả tóm tắt 41
Ảnh: Giao diện cài đặt tài khoản 42
Ảnh: Giao diện quản lý hệ thống 42

CHƯƠNG 1: GIỚI THIỆU VỀ ĐỀ TÀI

Đề tài "Hệ thống Tóm tắt và Trích xuất thông tin Văn bản Tiếng Việt Ứng dụng AI" (SummVi) được xây dựng nhằm giải quyết bài toán trên. Hệ thống cung cấp nền tảng web trực quan, cho phép người dùng tải lên văn bản hoặc tài liệu (PDF, Word) kết hợp với việc chọn đặc điểm cho văn bản đầu ra và nhận lại ngay các bản tóm tắt ngắn gọn, đầy đủ ý chính, bám sát nguyên tác chỉ trong vài giây.

1. Giải pháp công nghệ

- Áp dụng Kiến trúc RAG (Retrieval-Augmented Generation) để truy xuất nội dung tài liệu gốc, giúp giảm thiểu hallucination (ảo giác) của mô hình sinh.

- Sử dụng lõi mô hình ngôn ngữ ViT5 được tinh chỉnh chuyên biệt cho việc đọc hiểu và tóm tắt tiếng Việt.

- Xây dựng hệ thống Backend tốc độ cao bằng FastAPI, kết hợp cơ sở dữ liệu PostgreSQL (lưu trữ thông tin người dùng) và ChromaDB (lưu trữ vector ngữ nghĩa).

2. Ý nghĩa thực tiễn

Dự án hướng tới mục tiêu tối ưu hóa thời gian tiếp nhận thông tin. Sản phẩm là công cụ đắc lực hỗ trợ cho sinh viên, nhà nghiên cứu và giới văn phòng quản lý dữ liệu số một cách hiệu quả, đóng góp trực tiếp vào quá trình tăng cường hiệu suất làm việc trong kỷ nguyên thông tin.

CHƯƠNG 2: THIẾT KẾ XÂY DỰNG

1. Xác nhận yêu cầu hệ thống

1.1. Sơ đồ mô hình hoạt động của hệ thống

Quy trình hoạt động của hệ thống được mô tả như sau:

Bước 1: Người dùng truy cập hệ thống thông qua giao diện web.

Bước 2: Kiểm tra xem người dùng đã thực hiện đăng nhập/đăng ký chưa. Nếu chưa thì sang bước 3, nếu thực hiện rồi thì sang bước 4.

Bước 3: Người dùng thực hiện đăng nhập hoặc đăng ký.

Bước 4: Sau khi xác thực thành công, người dùng thực hiện một trong các hành động:

+ Nhập văn bản (sang bước 6).

+ Upload tài liệu (sang bước 5).

Bước 5: Hiển thị đoạn văn bản đã được upload lên giao diện chỉnh sửa văn bản.

Bước 6: Người dùng chọn loại tóm tắt.

Bước 7: Backend nhận request và chuyển đến AI Processing Module.

Bước 8: Hệ thống thực hiện pipeline RAG:

+ Tách văn bản (Chunking)

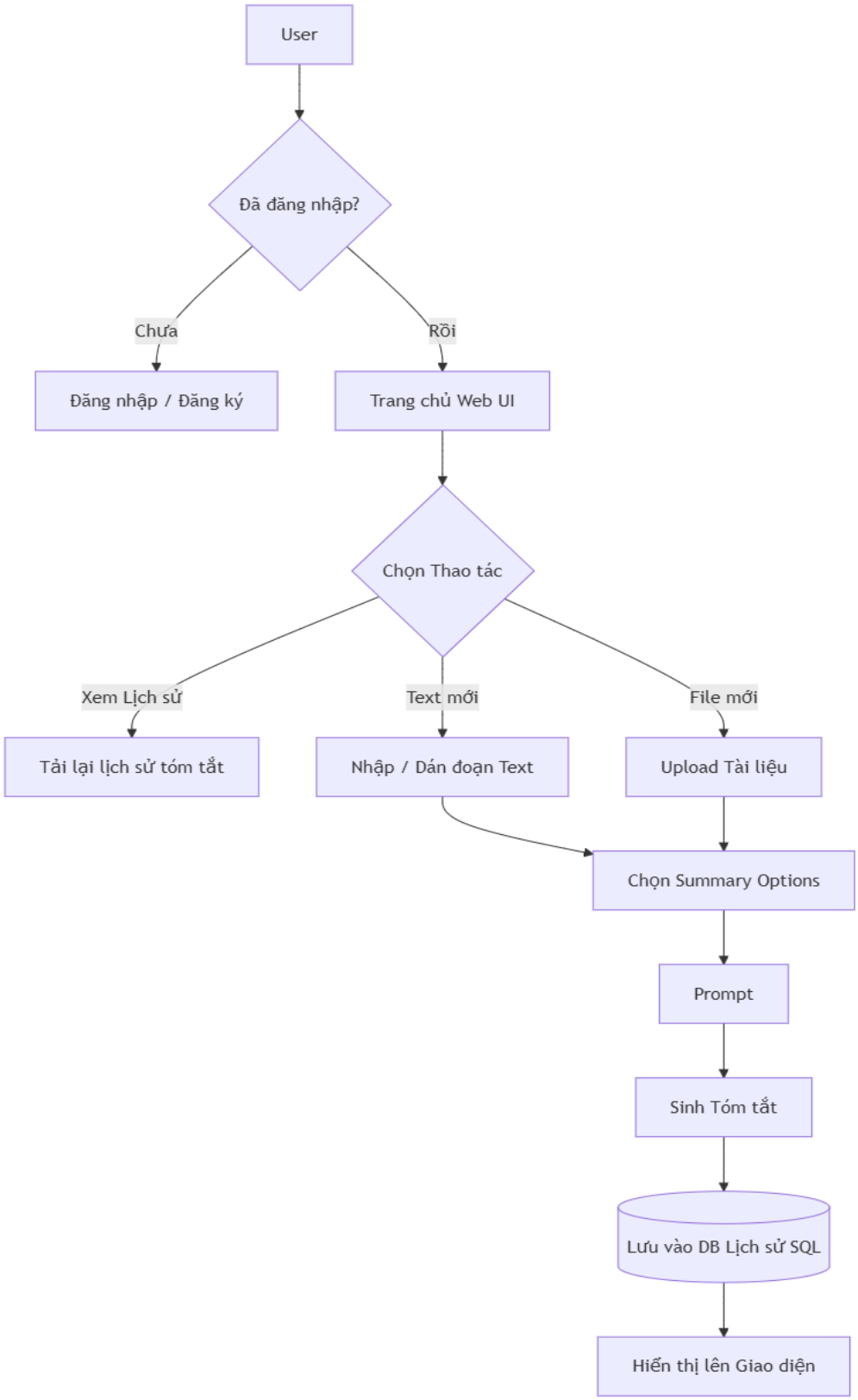
+ Sinh embedding (Sử dụng mô hình embedding)

+ Truy vấn vector database

+ Sinh kết quả tóm tắt

Bước 9: Kết quả được lưu vào cơ sở dữ liệu.

Bước 10: Backend trả kết quả về frontend để hiển thị cho người dùng.



Ảnh: Luồng người dùng cơ bản

1.2. Mô tả yêu cầu hệ thống

Yêu cầu chức năng

- Hệ thống cho phép người dùng đăng ký và đăng nhập tài khoản
- Cho phép người dùng nhập trực tiếp văn bản hoặc tải lên tài liệu (PDF, DOCX)
- Hệ thống thực hiện tóm tắt văn bản tự động bằng AI (RAG)
- Hiển thị kết quả tóm tắt cho người dùng
- Lưu trữ lịch sử tóm tắt để người dùng có thể xem lại
- Cho phép người dùng xóa lịch sử khi cần
- Hỗ trợ người dùng đánh giá chất lượng kết quả (rating, feedback)
- Cung cấp chức năng cho admin:

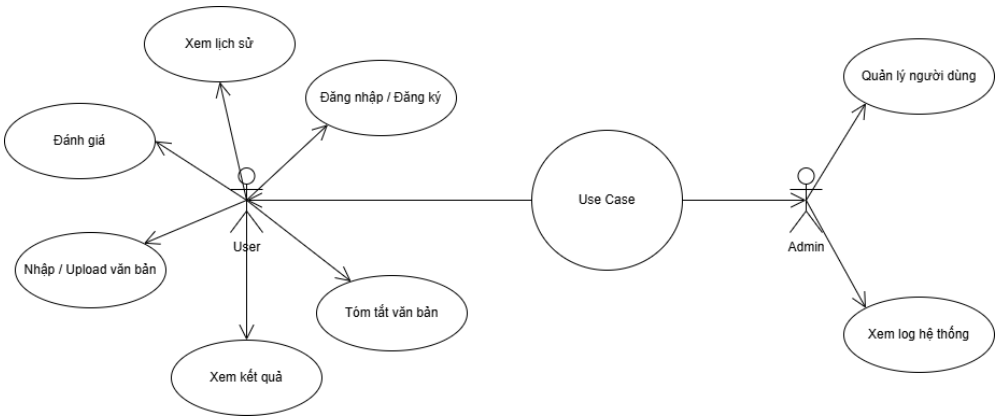
    + Quản lý người dùng

    + Xem log hệ thống

Yêu cầu phi chức năng

- Hệ thống phải có thời gian xử lý nhanh, phản hồi trong vài giây
- Đảm bảo tính bảo mật (xác thực bằng JWT, phân quyền người dùng/admin)
- Có khả năng mở rộng khi số lượng người dùng tăng
- Đảm bảo tính ổn định và sẵn sàng cao khi vận hành
- Hỗ trợ lưu trữ dữ liệu lớn (văn bản và vector embedding)
- Giao diện thân thiện, dễ sử dụng

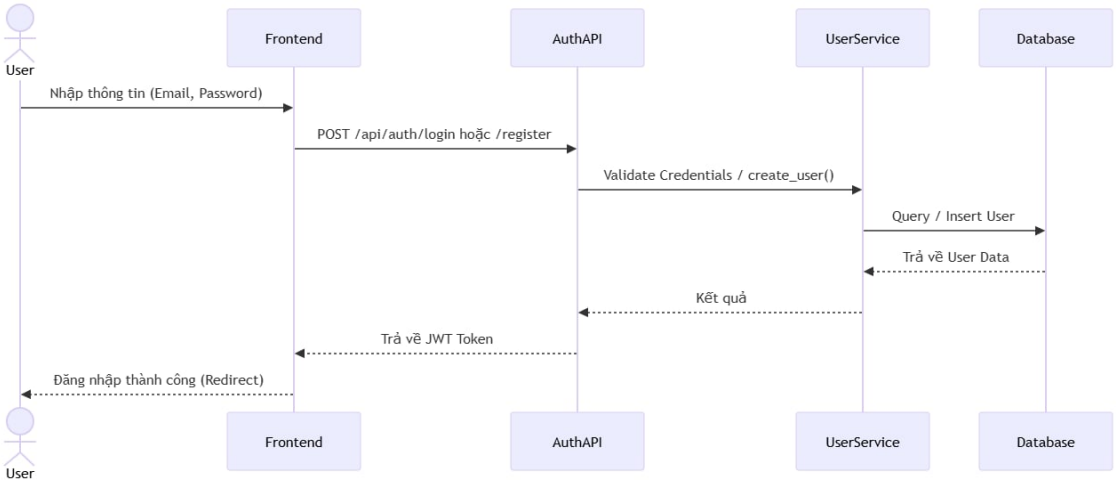
1.3. Biểu đồ Usecase



Ảnh: Usecase tổng quát

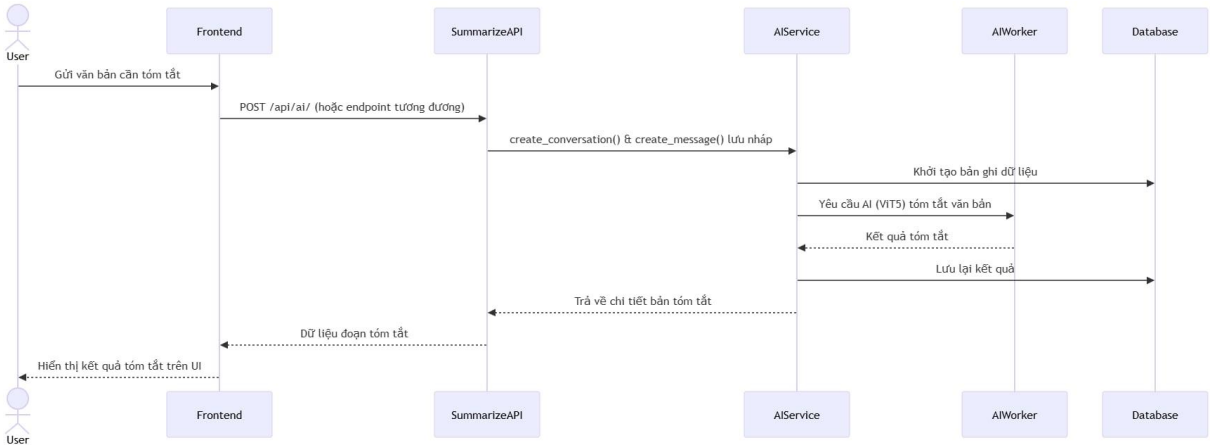
1.4. Biểu đồ tuần tự giao tiếp

1.4.1 Biểu đồ tuần tự đăng ký/đăng nhập



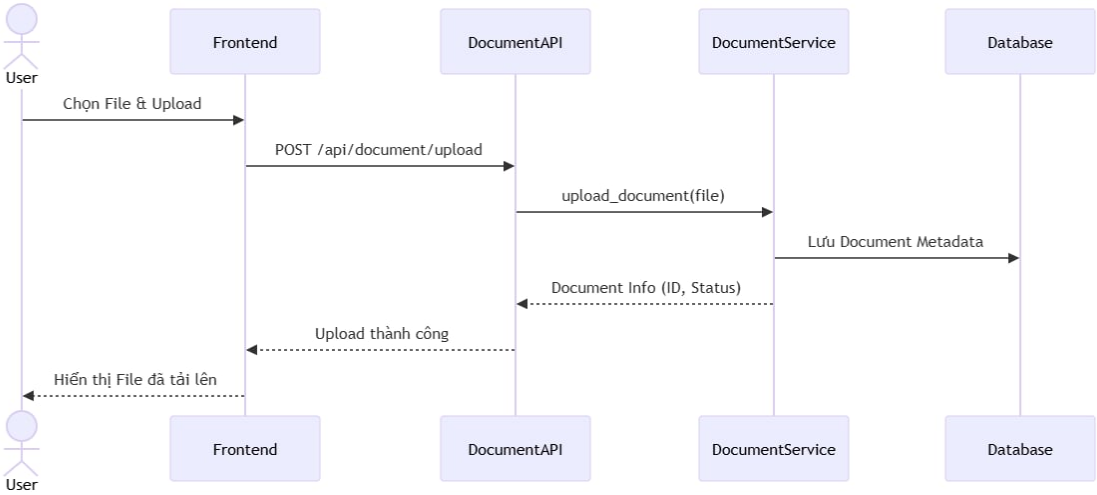
Ảnh: Biểu đồ tuần tự đăng ký/đăng nhập

1.4.2 Biểu đồ tuần tự tóm tắt văn bản



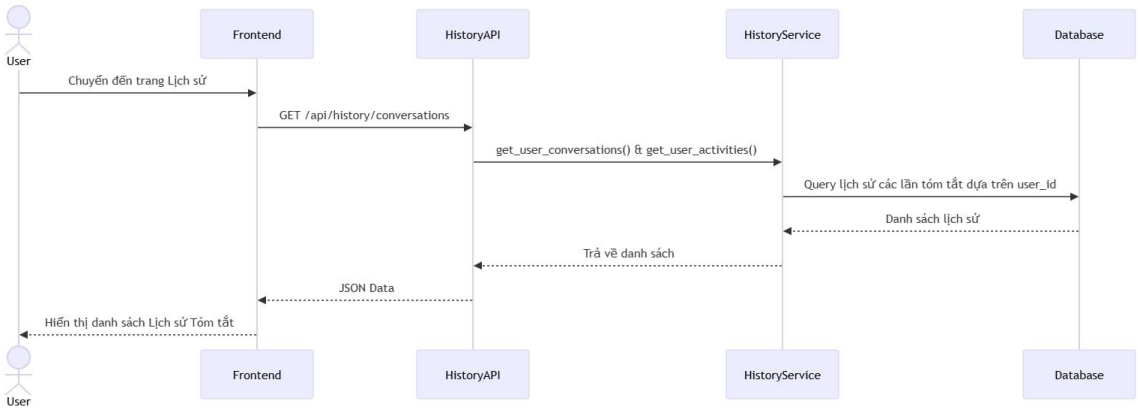
Ảnh: Biểu đồ tuần tự tóm tắt văn bản

1.4.3 Biểu đồ upload tài liệu



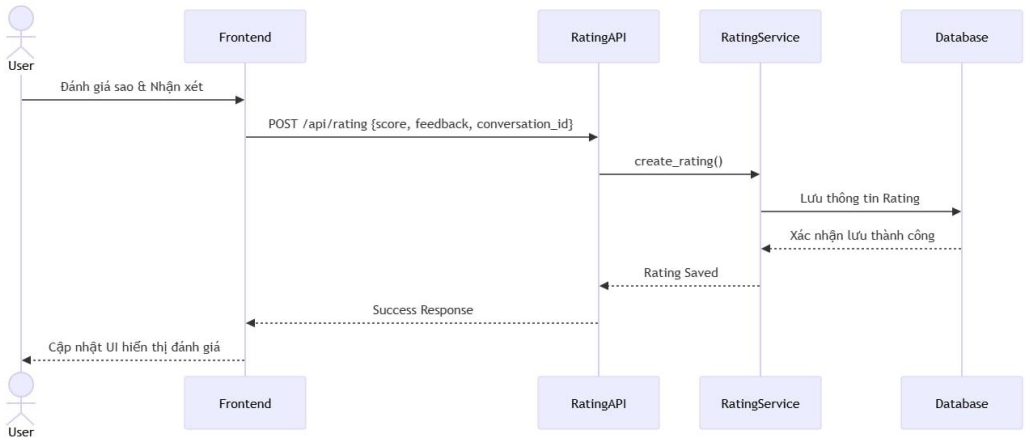
Ảnh: Biểu đồ tuần tự upload tài liệu

1.4.4 Biểu đồ tuần tự xem lịch sử tóm tắt



Ảnh: Biểu đồ tuần tự xem lịch sử tóm tắt

1.4.5 Biểu đồ tuần tự đánh giá



Ảnh: Biểu đồ tuần tự đánh giá

2. Phân tích hệ thống

2.1. Mô tả phân tích hệ thống

Hệ thống được chia thành các thành phần chính như sau:

2.1.1. Thành phần frontend (giao diện người dùng)

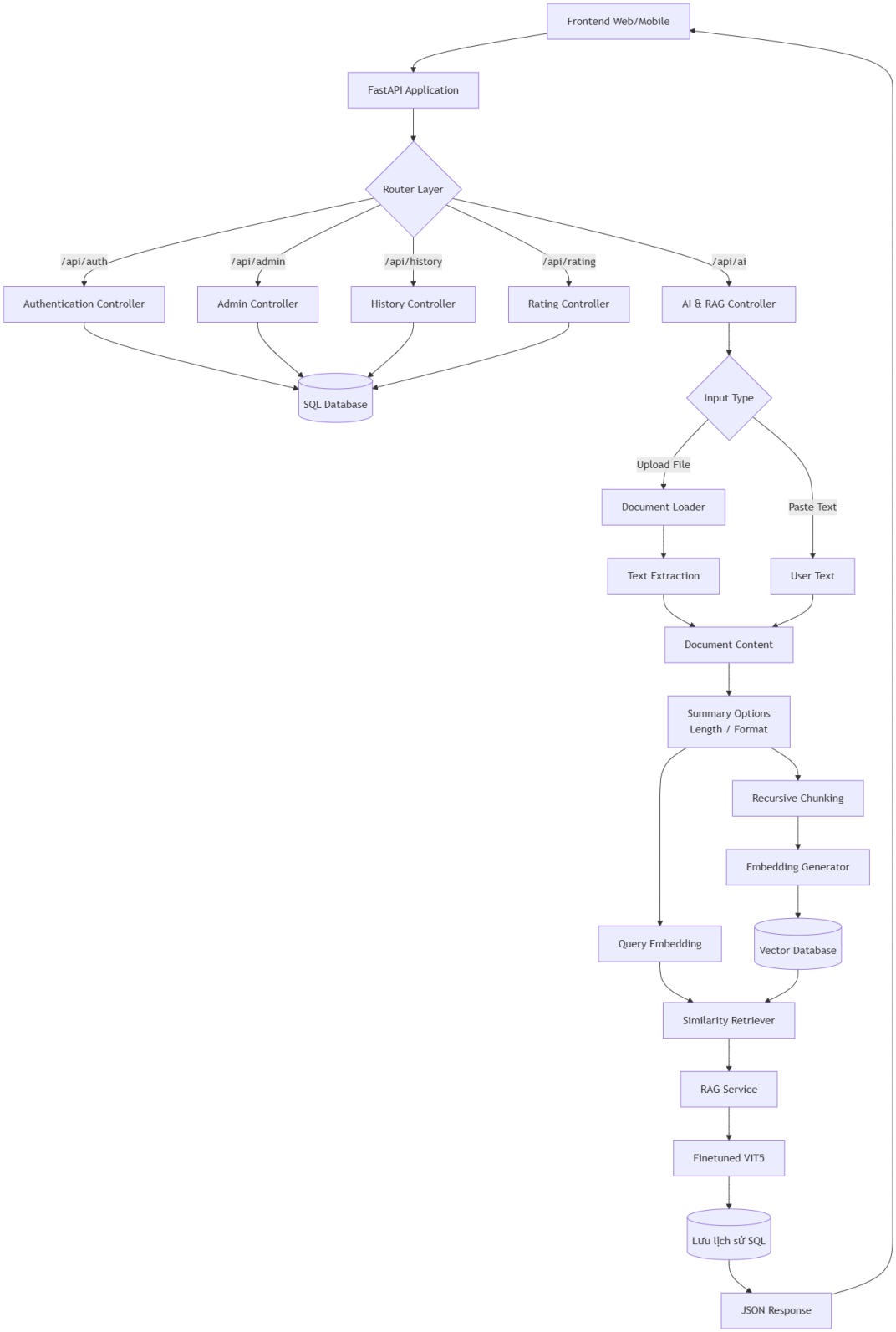
Thành phần này đóng vai trò:

- Cung cấp giao diện tương tác cho người dùng
- Gửi yêu cầu thông qua REST API đến backend
- Hiển thị kết quả tóm tắt

2.1.2. Thành phần backend

Backend đóng vai trò trung tâm trong hệ thống:

- Tiếp nhận và xử lý request từ client
- Điều phối các module chức năng
- Giao tiếp với cơ sở dữ liệu và AI module



Ảnh: Kiến trúc Server (Backend) tổng thể

2.1.2.1. Authentication (Xác thực)

- Chức năng: Giúp người dùng có thể đăng ký hoặc đăng nhập một tài khoản, đồng thời xác thực người dùng bằng JWT (JSON Web Token).

- Dữ liệu: Lưu trong cơ sở dữ liệu quan hệ PostreSQL.

- Đầu ra: mã truy cập hay một chuỗi kí tự đặc biệt hoạt động như "*chìa khóa tạm thời*" dùng để xác thực quyền truy cập của người dùng.

2.1.2.2. AI Core (Lõi AI)

- Chức năng: nhận input là một file văn bản hoặc văn bản do người dùng gõ trực tiếp rồi xử lý văn bản thông quan luồng RAG và cuối cùng sinh ra văn bản đã được tóm tắt.

- Các thành phần:

    + Text extractor

    + Chunking

    + Embedding

    + Retrieval

    + Generation (ViT5)

2.1.2.3. History (Lịch sử)

- Chức năng: Lưu trữ các kết quả tóm tắt và truy xuất lại các văn bản đã được tóm tắt bởi người dùng.

2.1.2.4. Admin (Quản trị viên)

- Chức năng: quản lý và theo dõi người dùng, theo dõi hoạt động của hệ thống và xem log (bản ghi lưu trữ quá trình xử lý).

2.1.2.5. Rating (Đánh giá)

- Chức năng: Thu thập đánh giá từ người dùng

2.1.2.6. Logging & Monitoring (Nhật ký và giám sát)

- Chức năng: ghi nhận các yêu cầu (request), theo dõi lỗi (error) và phân tích hiệu năng của hệ thống (performance).

2.1.3 Thành phần cơ sở dữ liệu

Cơ sở dữ liệu trong hệ thống tóm tắt văn bản sẽ đảm nhận vai trò lưu trữ, xử lý và truy xuất thông tin. Do đặc thù của hệ thống bao gồm dữ liệu có cấu trúc và dữ liệu phi cấu trúc nên nhóm lựa chọn mô hình lưu trữ sau:

- Cơ sở dữ liệu quan hệ (Relational Database).
- Cơ sở dữ liệu vectơ (Vecto Database).

Việc sử dụng hai loại cơ sở dữ liệu cho phép hệ thống:

- Quản lý hiệu quả dữ liệu người dùng, lịch sử đoạn tóm tắt và các thông số của hệ thống (dữ liệu có cấu trúc).
- Hỗ trợ tìm kiếm ngữ nghĩa thông qua các embedding (dữ liệu phi cấu trúc).

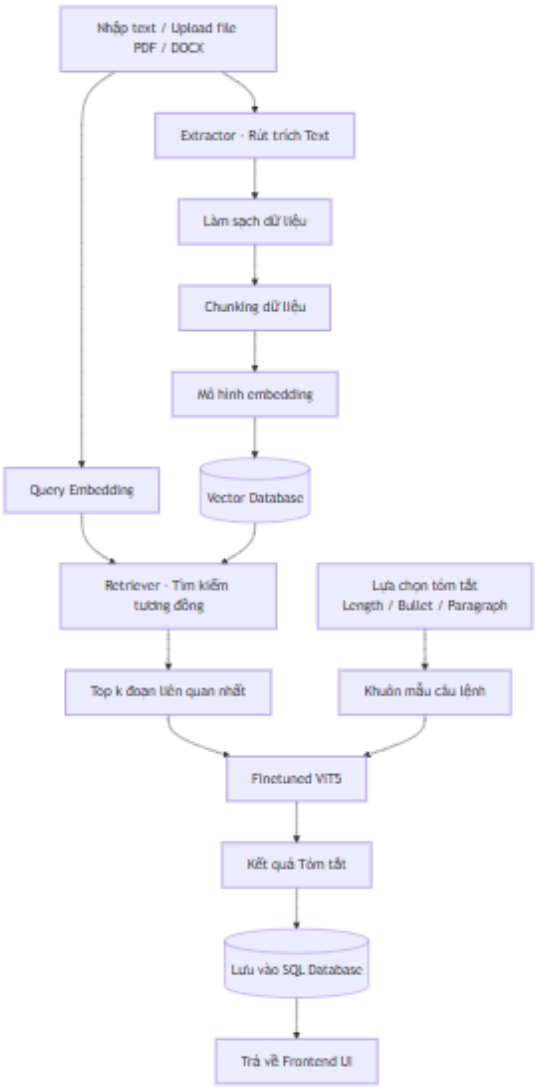
PostgreSQL (một hệ quản trị cơ sở dữ liệu quan hệ) được sử dụng để lưu trữ dữ liệu có cấu trúc như người dùng, hội thoại, lịch sử tóm tắt, thông số hệ thống.

ChromaDB (một cơ sở dữ liệu lưu trữ và sử dụng vectơ nhúng) được sử dụng để lưu trữ vector embedding của văn bản phục vụ cho hệ thống RAG.

2.1.4 Thành phần xử lý AI (lõi)

2.1.4.1. Sơ đồ hệ thống lõi

Hệ thống sử dụng pipeline RAG kết hợp với mô hình ViT5 đã finetune với mục đích tóm tắt:



Ảnh: Luồng xử lý RAG

Thành phần xử lý AI bao gồm:

- Text Extraction (Trích xuất văn bản)
- Text Preprocessing (Tiền xử lý dữ liệu)
- Text Chunking (Chia nhỏ văn bản)
- Embedding Generation (Sinh vector biểu diễn)
- Vector Database (Lưu trữ vector)
- Retriever (Truy xuất ngữ nghĩa)
- Context Builder (Xây dựng ngữ cảnh)
- Prompt Template (Tạo câu lệnh cho mô hình)
- Language Model (ViT5)

2.1.4.2. Quy trình xử lý

Quy trình xử lý RAG được chia làm 2 giai đoạn: Indexing và Retrieval & Generation

2.1.4.2.1 Giai đoạn Indexing (Xây dựng dữ liệu)

Giai đoạn này được thực hiện khi người dùng cung cấp tài liệu đầu vào.

Bước 1: Trích xuất văn bản (Text Extraction)

- Hệ thống tiếp nhận dữ liệu đầu vào dưới dạng: Văn bản thuần hoặc file PDF/DOCX
- Nội dung được trích xuất thành văn bản thô để xử lý tiếp theo

Bước 2: Tiền xử lý dữ liệu (Text Preprocessing)

- Loại bỏ ký tự không cần thiết
- Chuẩn hóa định dạng văn bản
- Xử lý xuống dòng và khoảng trắng

Bước 3: Chia nhỏ văn bản (Chunking)

- Sử dụng phương pháp Recursive Chunking
- Văn bản được chia thành các đoạn nhỏ có độ dài phù hợp

Bước 4: Sinh embedding

- Mỗi đoạn văn bản được chuyển thành vector thông qua embedding model
- Vector thể hiện ngữ nghĩa của đoạn văn

Bước 5: Lưu trữ vào Vector Database

- Các vector được lưu vào hệ quản trị vector (ChromaDB)
- Kèm theo metadata (nguồn, vị trí đoạn,…)

Ảnh: Minh họa giai đoạn Indexing

2.1.4.2.2 Giai đoạn Retrieval và Generation

Bước 1: Xử lý truy vấn (Query Processing)

- Input của người dùng được chuyển thành vector embedding
- Vector này dùng để tìm kiếm ngữ nghĩa

Bước 2: Truy xuất dữ liệu (Retrieval)

- Hệ thống sử dụng phương pháp similarity search
- Truy xuất Top-k đoạn văn bản liên quan nhất

Bước 3: Xây dựng ngữ cảnh (Context Builder)

- Các đoạn văn bản được kết hợp thành một ngữ cảnh hoàn chỉnh
- Ngữ cảnh này sẽ được đưa vào mô hình sinh

Bước 4: Tạo Prompt (Prompt Template): kết hợp ngữ cảnh và tùy chọn của người dùng tạo thành câu lệnh hoàn chỉnh gửi vào mô hình.

Bước 5: Sinh kết quả (Generation): mô hình ViT5 được sử dụng để sinh nội dung tóm tắt

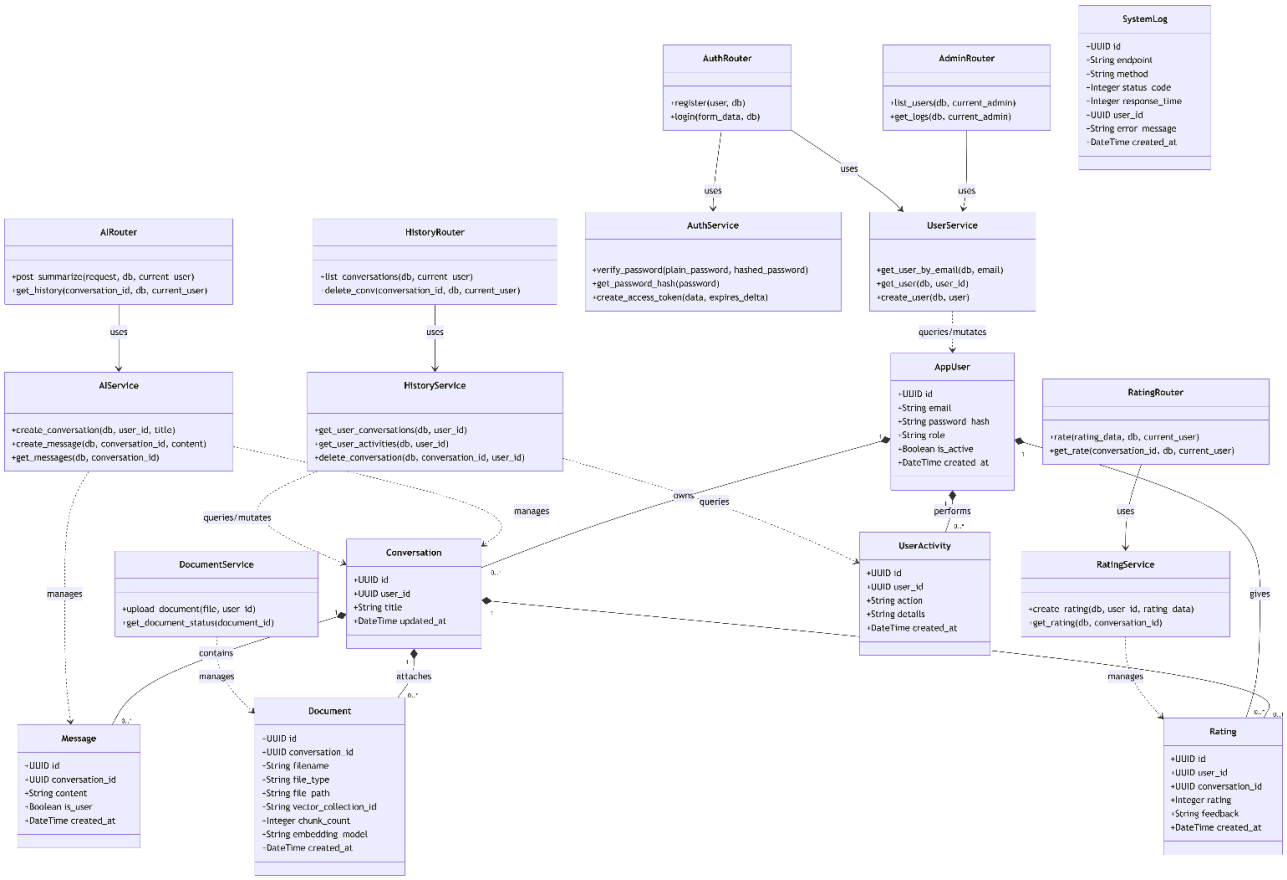
Bước 6: Lưu trữ kết quả vào cơ sở dữ liệu quan hệ và trả kết quả ở frontend.

2.1.5. Thành phần phân tích dữ liệu (BI)

Hệ thống hỗ trợ phân tích dữ liệu thông qua PowerBI:

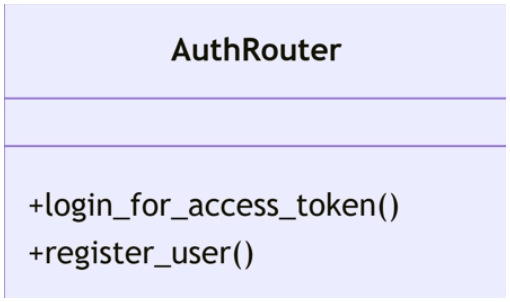
- Kết nối trực tiếp với cơ sở dữ liệu SQL
- Trực quan hóa: hiệu năng hệ thống, tỷ lệ lỗi ,đánh giá người dùng,…

2.2. Biểu đồ lớp (Class Diagram)



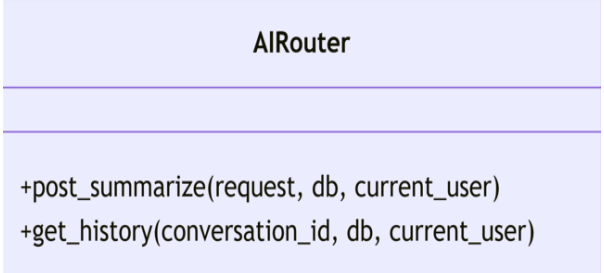
Ảnh: Biểu đồ lớp tổng quát

2.2.1 AuthRouter



Thành phần	**Mô tả**
Định nghĩa	Lớp chịu trách nhiệm xử lý các yêu cầu liên quan đến xác thực người dùng, bao gồm đăng ký và đăng nhập.
Thuộc tính	
Phương thức	+ register(): Đăng ký tài khoản người dùng + login(): Đăng nhập vào hệ thống

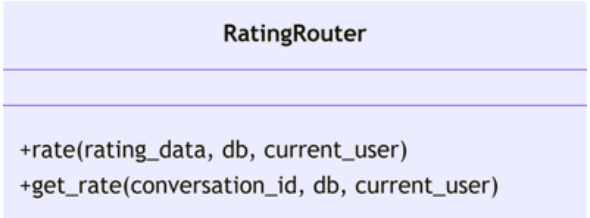
2.2.2 AIRouter



Thành phần	**Mô tả**

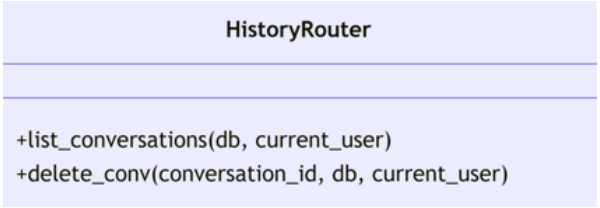
Định nghĩa	Lớp xử lý các yêu cầu liên quan đến chức năng AI như tóm tắt văn bản và truy xuất lịch sử.
Thuộc tính	
Phương thức	+ post_summarize(request, db, current_user): Tóm tắt văn bản + get_history(conversation_id, db, current_user): Lấy lịch sử

2.2.3 RatingRouter



Thành phần	Mô tả
Định nghĩa	Lớp xử lý các yêu cầu đánh giá từ người dùng.
Thuộc tính	
Phương thức	+ rate(rating_data, db, current_user): Gửi đánh giá + get_rate(conversation_id, db, current_user): Lấy đánh giá

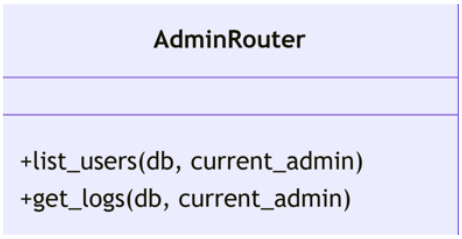
2.2.4 HistoryRouter



Thành phần	Mô tả

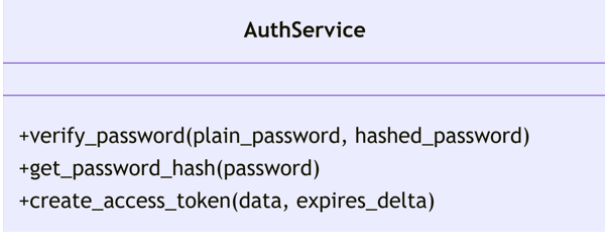
Định nghĩa	Lớp quản lý lịch sử hội thoại của người dùng.
Thuộc tính	
Phương thức	+ list_conversations(db, current_user): Danh sách hội thoại + delete_conv(conversation_id, db, current_user): Xóa hội thoại

2.2.5 AdminRouter



Thành phần	Mô tả
Định nghĩa	Lớp xử lý các chức năng quản trị hệ thống.
Thuộc tính	
Phương thức	+ list_users(db, current_admin): Danh sách người dùng + get_logs(db, current_admin): Lấy log hệ thống

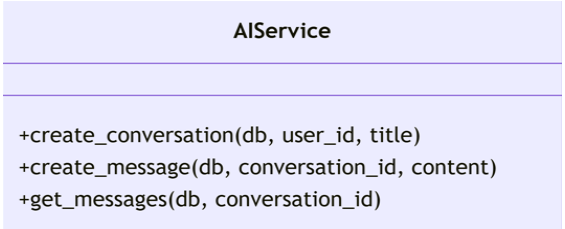
2.2.6 AuthService



Thành phần	Mô tả

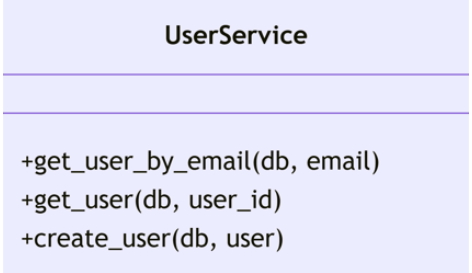
Định nghĩa	Xử lý các chức năng xác thực và bảo mật người dùng.
Thuộc tính	
Phương thức	+ verify_password(plain_password, hashed_password): Xác thực mật khẩu người dùng + get_password_hash(password): Mã hóa mật khẩu trước khi lưu + create_access_token(data, expires_delta): Tạo token truy cập

2.2.7 AIService



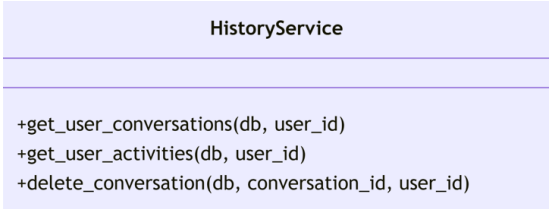
Thành phần	**Mô tả**
Định nghĩa	Điều phối toàn bộ quy trình xử lý AI và quản lý hội thoại.
Thuộc tính	
Phương thức	+ create_conversation(db, user_id, title): Tạo hội thoại mới + create_message(db, conversation_id, content): Lưu nội dung trao đổi + get_messages(db, conversation_id): Lấy danh sách message

2.2.8 UserService



Thành phần	Mô tả
Định nghĩa	Quản lý thông tin và thao tác liên quan đến người dùng.
Thuộc tính	
Phương thức	+ get_user_by_email(db, email): Tìm người dùng bằng email + get_user(db, user_id): Lấy thông tin người dùng bằng ID + create_user(db, user): Tạo tài khoản người dùng mới

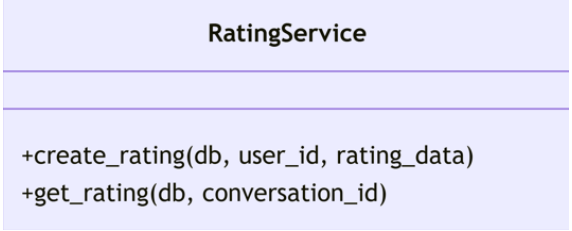
2.2.9 HistoryService



Thành phần	Mô tả
Định nghĩa	Xử lý các chức năng liên quan đến lịch sử và hoạt động người dùng.
Thuộc tính	
Phương thức	+ get_user_conversations(db, user_id): Lấy lịch sử hội thoại + get_user_activities(db, user_id): Lấy nhật ký hoạt động

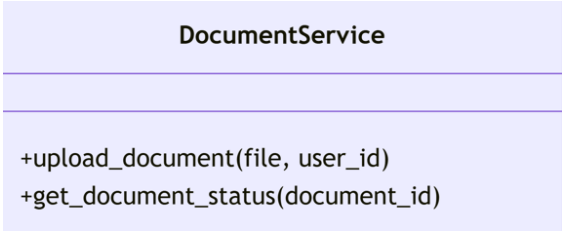
	+ delete_conversation(db, conversation_id, user_id): Xóa một cuộc trò chuyện

2.2.10 RatingService



Thành phần	**Mô tả**
Định nghĩa	Quản lý việc lưu và truy xuất đánh giá của người dùng.
Thuộc tính	
Phương thức	+ create_rating(db, user_id, rating_data): Lưu đánh giá mới. + get_rating(db, conversation_id): Lấy đánh giá của hội thoại.

2.2.11 DocumentService

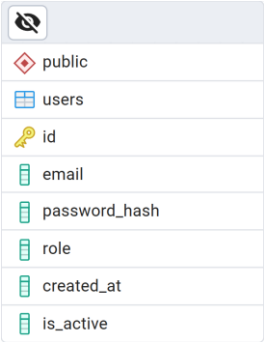


Thành phần	**Mô tả**
Định nghĩa	Quản lý tài liệu được tải lên và trạng thái xử lý.
Thuộc tính	
Phương thức	+ upload_document(file, user_id): Tải tài liệu lên

	+ get_document_status(document_id): Xem trạng thái xử lý

2.2.12 User

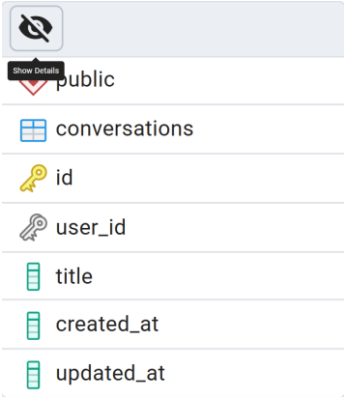
Lưu thông tin người dùng



Trường	**Kiểu dữ liệu**	**Mô tả**
id	UUID (PK)	Định danh duy nhất
email	VARCHAR	Email đăng nhập
password_hash	VARCHAR	Mật khẩu mã hóa
role	VARCHAR	Vai trò người dùng
created_at	TIMESTAMP	Thời điểm tạo
is_active	BOOLEAN	Trạng thái tài khoản

2.2.13 Conversation

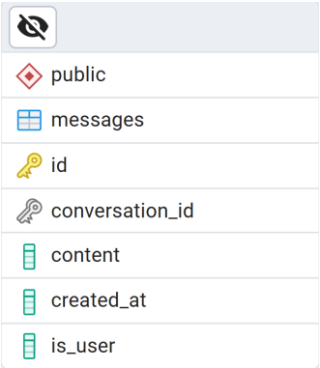
Đại diện cho phiên làm việc



Trường	Kiểu dữ liệu	Mô tả
id	UUID (PK)	Định danh hội thoại
user_id	UUID (FK)	Liên kết USERS
title	VARCHAR	Tiêu đề hội thoại
created_at	TIMESTAMP	Thời điểm tạo
updated_at	TIMESTAMP	Thời điểm cập nhật

2.2.14 Message

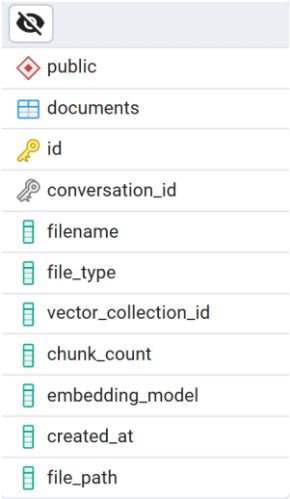
Lưu nội dung hội thoại



Trường	Kiểu dữ liệu	Mô tả
id	UUID (PK)	ID message
conversation_id	UUID (FK)	Liên kết CONVERSATIONS
content	TEXT	Nội dung
is_user	BOOLEAN	Xác định nguồn gửi
created_at	TIMESTAMP	Thời gian

2.2.15 Document

Lưu thông tin tài liệu và embedding

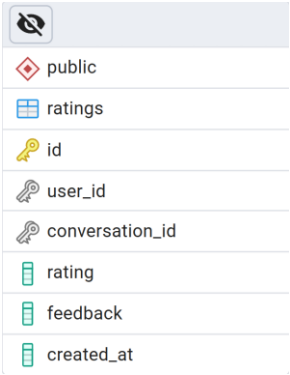


Trường	Kiểu dữ liệu	Mô tả
id	UUID (PK)	ID tài liệu
conversation_id	UUID (FK)	Liên kết CONVERSATIONS

filename	VARCHAR	Tên file
file_type	VARCHAR	Loại file
file_path	VARCHAR	Đường dẫn lưu trữ
vector_collection_id	VARCHAR	ID trong vector DB
chunk_count	INT	Số đoạn
embedding_model	VARCHAR	Mô hình embedding
created_at	TIMESTAMP	Thời gian

2.2.16 Rating

Lưu đánh giá

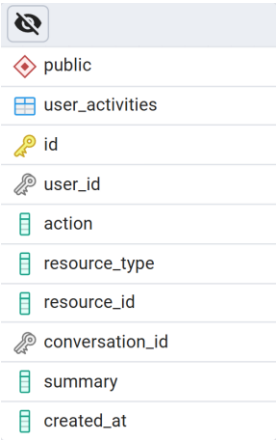


Trường	**Kiểu dữ liệu**	**Mô tả**
id	UUID (PK)	ID
user_id	UUID (FK)	Liên kết USERS
conversation_id	UUID (FK)	Liên kết CONVERSATIONS
rating	INT	Điểm đánh giá

feedback	TEXT	Nhận xét
created_at	TIMESTAMP	Thời gian

2.2.17 UserActivity

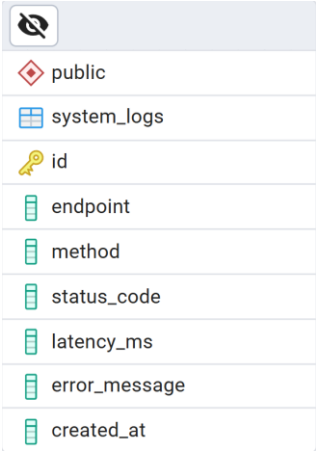
Lưu hoạt động người dùng



Trường	**Kiểu dữ liệu**	**Mô tả**
id	UUID (PK)	ID
user_id	UUID (FK)	Liên kết USERS
action	VARCHAR	Hành động
resource_type	VARCHAR	Loại tài nguyên
resource_id	UUID	ID tài nguyên
conversation_id	UUID (FK)	Liên kết CONVERSATIONS
summary	TEXT	Mô tả
created_at	TIMESTAMP	Thời gian

2.2.18 SystemLog

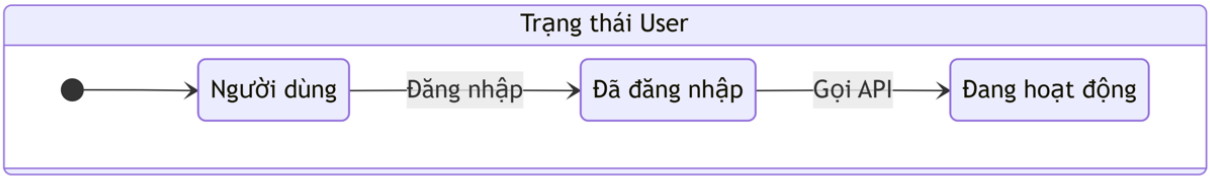
Ghi nhận log hệ thống



Trường	Kiểu dữ liệu	Mô tả
id	UUID (PK)	ID
endpoint	VARCHAR	API endpoint
method	VARCHAR	HTTP method
status_code	INT	Mã phản hồi
latency_ms	INT	Thời gian xử lý
error_message	TEXT	Nội dung lỗi
created_at	TIMESTAMP	Thời gian

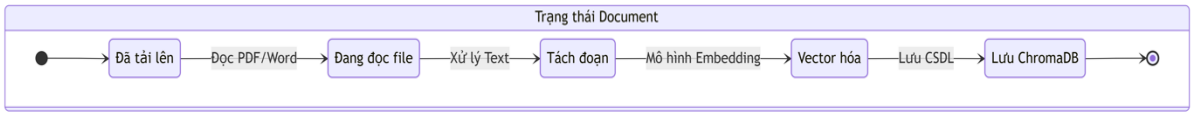
2.3 Biểu đồ trạng thái (State Diagram)

2.3.1 Trạng thái User



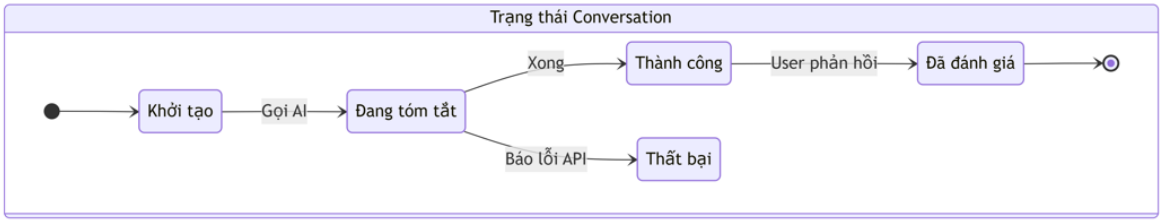
Ảnh: Biểu đồ trạng thái User

2.3.2 Trạng thái Document



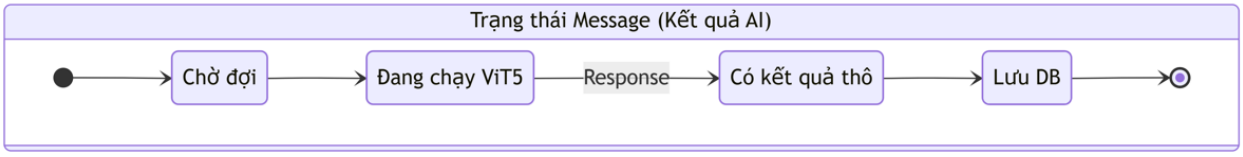
Ảnh: Biểu đồ trạng thái Document

2.3.3 Trạng thái Conversation



Ảnh: Biểu đồ trạng thái Conversation

3.2.5 Trạng thái Message



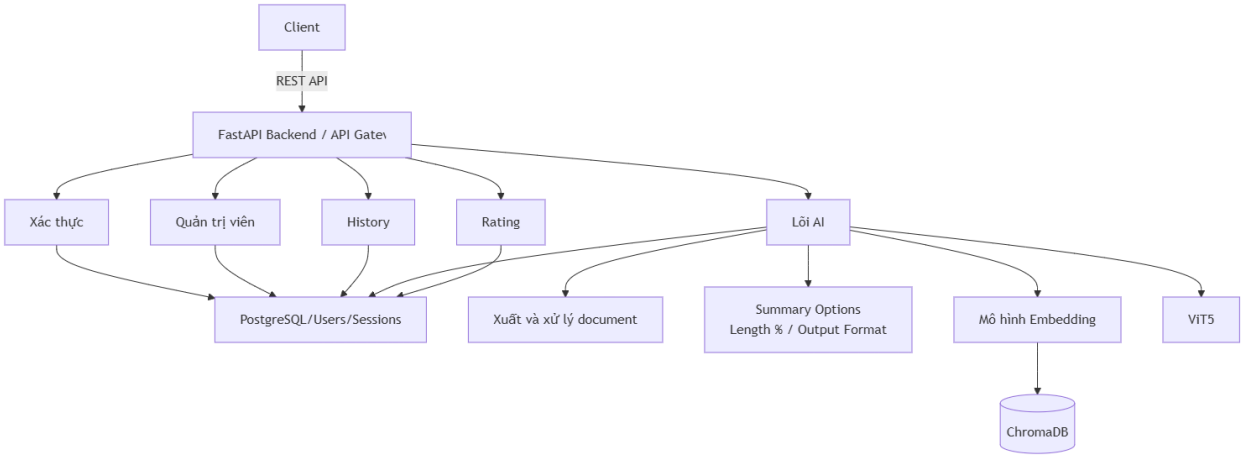
Ảnh: Biểu đồ trạng thái Message

3. Thiết kế hệ thống

3.1 Thiết kế kiến trúc hệ thống

Hệ thống được xây dựng theo mô hình Client – Server sử dụng kiến trúc 3 tầng nhằm đảm bảo tính mở rộng (Scalability), tính bảo mật (Security), tính bảo trì (Maintainance) và hiệu năng (Performance):

- Presentation (Frontend): Là giao diện người dùng. Tầng này sẽ không xử lý logic phức tạp mà có nhiệm vụ hiển thị giao diện và giao tiếp với Server thông qua REST API.
- Application (Backend): đóng vai trò xử lý toàn bộ nghiệp vụ cốt lõi (Business Logic), bao gồm: xác thực (Authentication), phân quyền (Admin), xử lý file, RAG Pipeline và giao tiếp với lõi AI (Mô hình ViT5 và Embedding).
- Data (Database): là nơi lưu trữ toàn bộ dữ liệu của hệ thống.



Ảnh: Kiến trúc tổng thể của hệ thống

3.2 Lựa chọn công nghệ cho hệ thống

3.2.1 Hệ quản trị cơ sở dữ liệu

PostgreSQL là một hệ thống cơ sở dữ liệu quan hệ đối tượng mã nguồn mở mạnh mẽ, sử dụng và mở rộng ngôn ngữ SQL kết hợp với nhiều tính năng giúp lưu trữ và mở rộng quy mô an toàn cho các khối lượng dữ liệu phức tạp nhất. Nguồn gốc của PostgreSQL bắt nguồn

từ năm 1986 như một phần của dự án POSTGRES tại Đại học California tại Berkeley và đã có gần 40 năm phát triển tích cực trên nền tảng cốt lõi.

Ảnh: PostgreSQL

Lý do sử dụng:

- Khả năng mở rộng cao: Postgres cung cấp khả năng mở rộng theo chiều ngang với các bản sao đọc và nhóm kết nối, điều này làm cho nó phù hợp với các ứng dụng có lưu lượng truy cập cao và độ đồng thời cao.
- Các tính năng nâng cao: Postgres có nhiều tính năng nâng cao như hỗ trợ JSON gốc, tìm kiếm toàn văn và hiệu suất tốt hơn cho các truy vấn phức tạp.
- Hiệu suất cao: Postgres nổi tiếng với hiệu suất cao, cho phép nó xử lý lượng lớn dữ liệu.

3.2.2 Cơ sở dữ liệu vectơ

Chroma DB là một cơ sở dữ liệu vector mã nguồn mở được thiết kế để lưu trữ, tìm kiếm và quản lý hiệu quả các vector nhúng – các biểu diễn số được sử dụng trong trí tuệ nhân tạo và máy học cho các tác vụ như tìm kiếm ngữ nghĩa và hệ thống đề xuất. Nó cho phép tìm kiếm sự tương đồng nhanh chóng và cung cấp API đơn giản cho các nhà phát triển, rất phù hợp để xây dựng và triển khai các ứng dụng dựa trên trí tuệ nhân tạo.

Ảnh: ChromaDB

Lý do sử dụng:

- Hỗ trợ nhiều ngôn ngữ lập trình: ChromaDB hỗ trợ nhiều ngôn ngữ lập trình khác nhau, giúp các nhà phát triển dễ dàng tích hợp nó vào hầu hết mọi nền tảng công nghệ.
- Dễ sử dụng: ChromaDB có thể chạy cục bộ; nó nhẹ và dễ cài đặt và thiết lập.
- API tìm kiếm thống nhất cho kết quả chi tiết: ChromaDB cung cấp API Tìm kiếm thống nhất và linh hoạt, đây là một thế mạnh cốt lõi để xây dựng các hệ thống RAG.
- Tính linh hoạt cho thử nghiệm và kiểm soát quy trình: Sự linh hoạt này đến từ khả năng cấu trúc các embedding, thêm siêu dữ liệu và thử nghiệm với quy trình truy vấn của bạn theo những cách mà một số cơ sở dữ liệu vector khác không dễ dàng cho phép.

3.2.2 Công cụ lập trình

Visual Studio Code: là trình soạn thảo mã nguồn (code editor) miễn phí, nhẹ và mạnh mẽ do Microsoft phát triển, hỗ trợ đa nền tảng (Windows, macOS, Linux).

Ảnh: Visual Studio Code

Lý do sử dụng:

- Hỗ trợ đa nền tảng bao gồm Windows, Linux và Mac.
- Khối lượng nhẹ.
- Cấu trúc mạnh mẽ.
- Intelli-Sense.
- Phần mềm miễn phí.
- Dành cho các ứng dụng máy tính.
- Hỗ trợ các công cụ công nghệ web như HTML, CSS, JSON.

3.2.3 Công nghệ lập trình phía server

FastAPI: là một framework web hiện đại, hiệu năng cao, chuyên dùng để xây dựng API với Python 3.7+ dựa trên các gợi ý kiểu dữ liệu (type hints) chuẩn.

Ảnh: FastAPI

Lý do sử dụng:

- Hiệu suất vượt trội: Được xây dựng dựa trên Starlette và Pydantic, FastAPI là một trong những framework Python nhanh nhất hiện nay, phù hợp cho các ứng dụng đòi hỏi hiệu năng cao.
- Phát triển nhanh: Giúp tăng tốc độ phát triển tính năng từ 200% đến 300%, giảm thời gian viết code.

- Tài liệu tự động: Tự động tạo tài liệu API tương tác (Swagger UI/ReDoc) ngay khi code, không cần viết tài liệu thủ công.
- Kiểm tra dữ liệu: Sử dụng Pydantic để tự động kiểm tra và xác thực dữ liệu đầu vào, giảm thiểu lỗi và mã nguồn sạch hơn.
- Hỗ trợ bất đồng bộ: Hỗ trợ async/await giúp xử lý đồng thời nhiều yêu cầu, tối ưu hóa cho các tác vụ I/O.
- Dễ sử dụng và trực quan: Thiết kế đơn giản, dễ học, tương tự như Flask nhưng tích hợp sẵn nhiều tính năng mạnh mẽ.
- Độ tin cậy và an toàn: Dựa trên gợi ý kiểu dữ liệu (type hints) của Python giúp giảm lỗi lập trình và tăng khả năng gợi ý mã (autocomplete) trên IDE.

3.2.4 Công nghệ lập trình phía client

Next.js: là một React framework mã nguồn mở được phát triển bởi Vercel, cho phép xây dựng ứng dụng web nhanh.

Ảnh: NextJS

Lý do sử dụng:

- Hiệu suất và SEO tối ưu: Với SSR và SSG, nội dung được render sẵn phía server, giúp trang web tải nhanh hơn, giảm tỷ lệ thoát và công cụ tìm kiếm dễ dàng cào dữ liệu hơn.
- Trải nghiệm Developer tuyệt vời: Định tuyến dựa trên cấu trúc thư mục/tập tin, giúp dễ quản lý. Hỗ trợ sẵn Webpack, Babel, TypeScript, và hot-reloading mà không cần cấu hình phức tạp.

- Dễ dàng triển khai: Tương thích hoàn hảo với Vercel (công ty phát triển Next.js) và các nền tảng khác như AWS, GCP, mang lại quy trình triển khai nhanh chóng.

3.2.5 Công nghệ quản lý source code

Github: là nền tảng đám mây lớn nhất thế giới giúp lập trình viên lưu trữ, quản lý và chia sẻ mã nguồn (source code) dựa trên hệ thống Git. Đây là công cụ thiết yếu để làm việc nhóm, cho phép nhiều người cùng chỉnh sửa dự án mà không xung đột, đồng thời hoạt động như một "mạng xã hội" để phát triển mã nguồn mở.

Ảnh: Github

Lý do sử dụng:

- Làm việc nhóm hiệu quả: Quản lý xung đột code khi nhiều người cùng làm việc.
- Lưu trữ an toàn & Miễn phí: Cung cấp không gian lưu trữ cho các dự án mã nguồn mở.

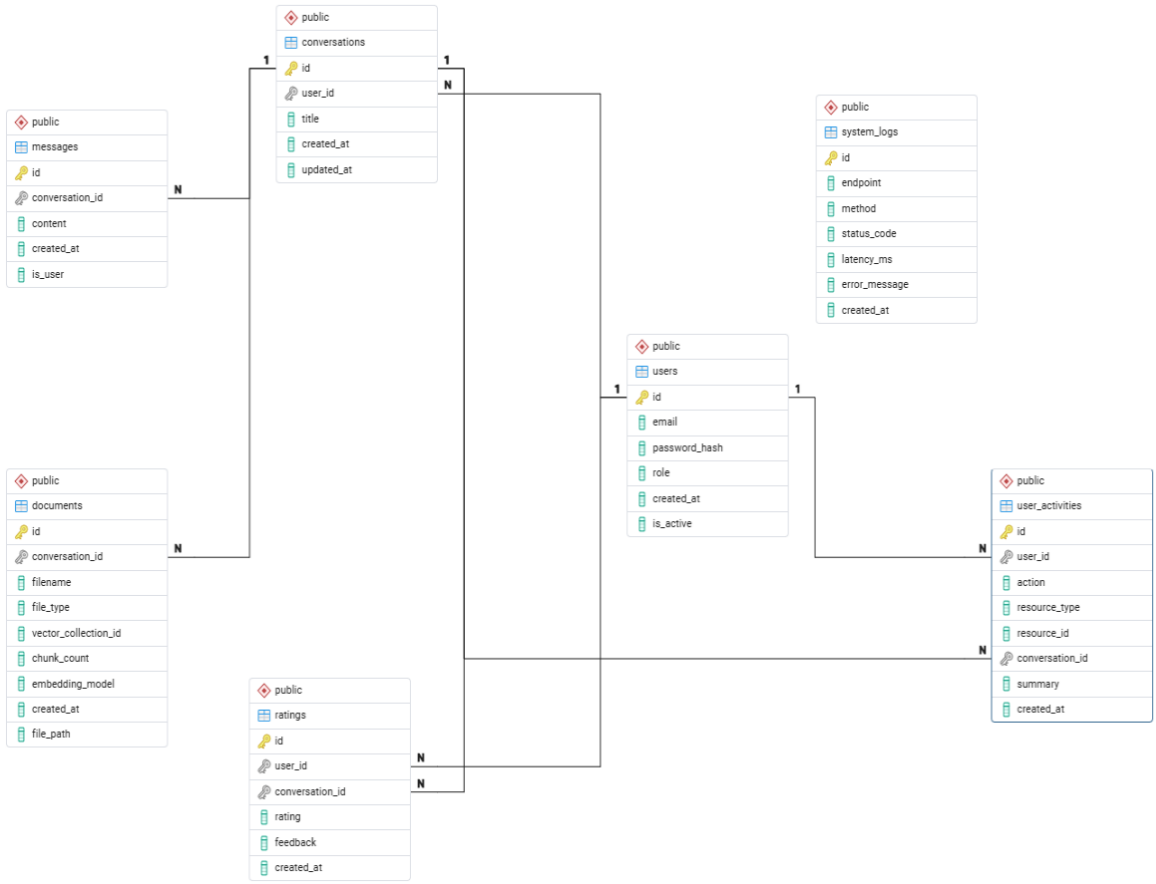
3.2.6 Công nghệ đóng gói

Docker: là nền tảng mã nguồn mở, giúp lập trình viên đóng gói, phân phối và chạy ứng dụng trong các container (bộ chứa) nhẹ, riêng biệt.

Ảnh: Docker

Lý do sử dụng: giúp đơn giản hóa quy trình cài đặt, nâng cao năng suất đội ngũ phát triển và tối ưu hóa tài nguyên phần cứng.

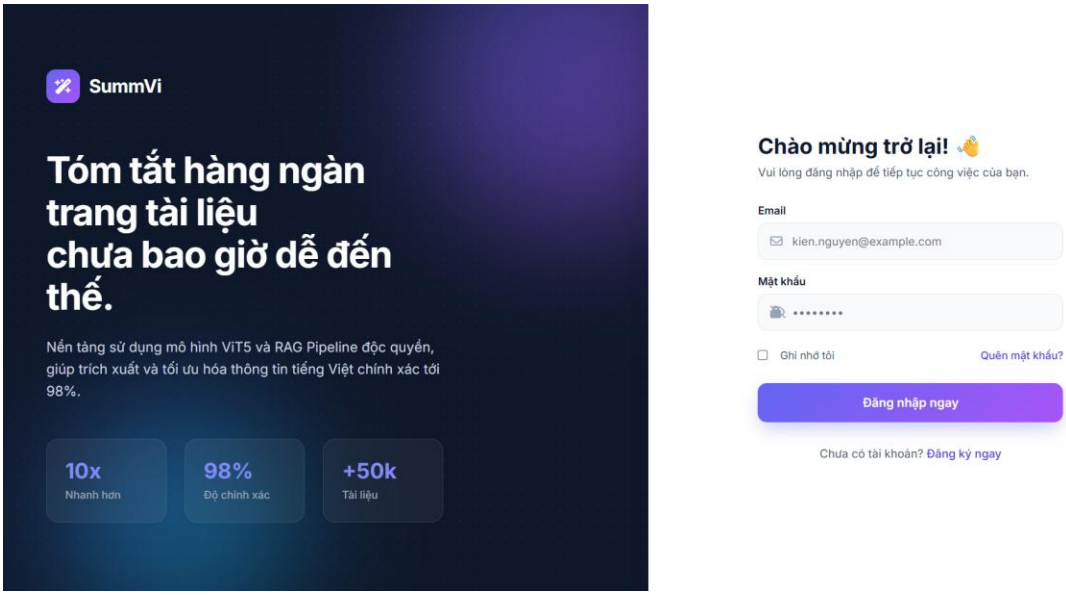
3.3 Thiết kế cơ sở dữ liệu



Ảnh: Biểu đồ ERD

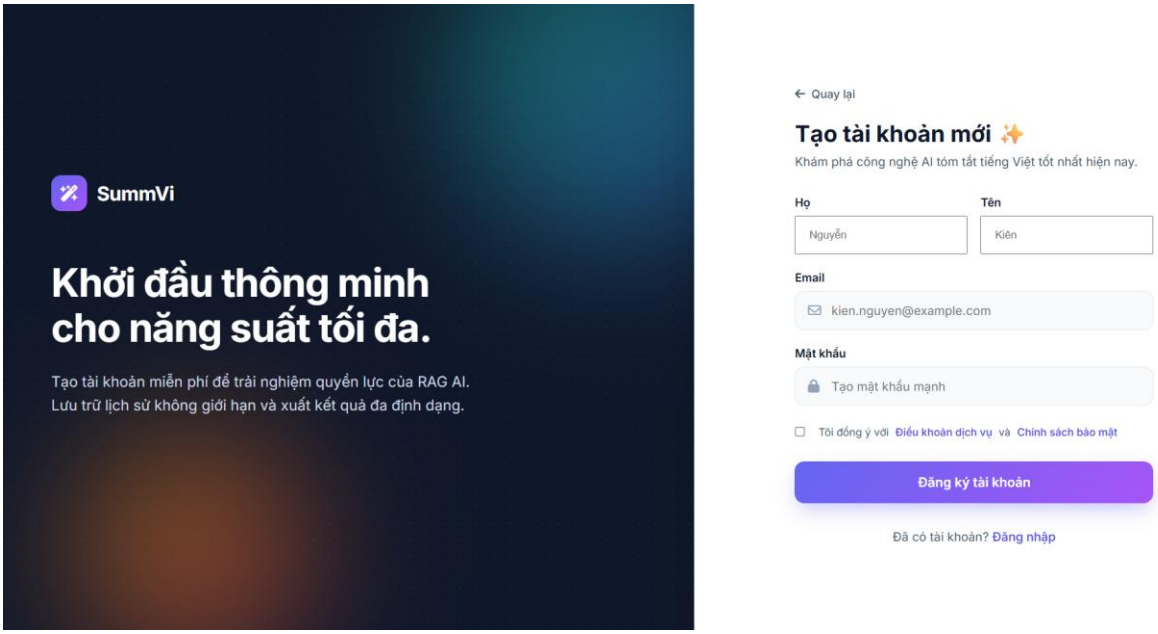
3.4 Thiết kế giao diện (Demo)

3.4.1 Giao diện đăng nhập



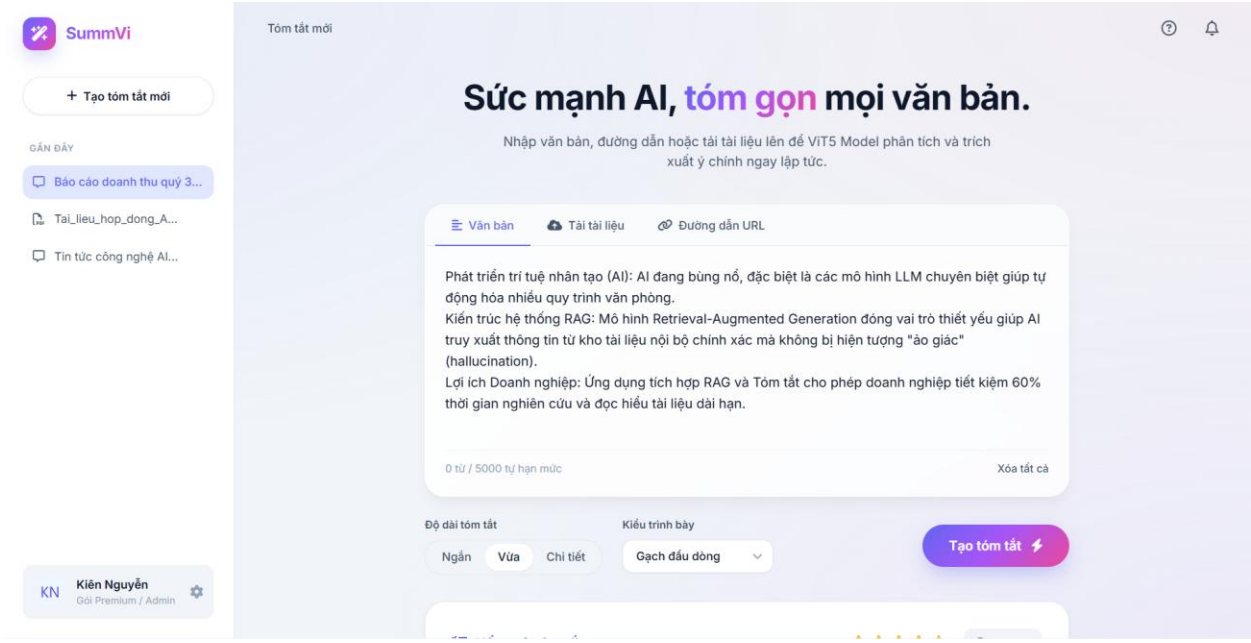
Ảnh: Giao diện đăng nhập

3.4.2 Giao diện đăng ký



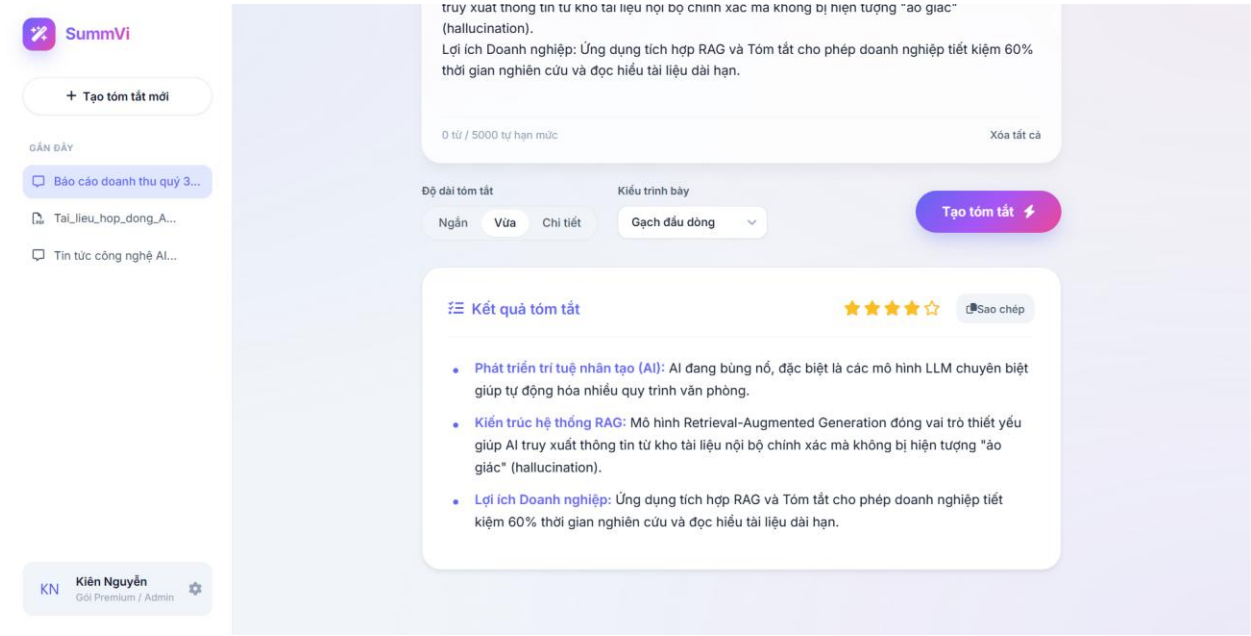
Ảnh: Giao diện đăng ký

3.4.3 Giao diện chính



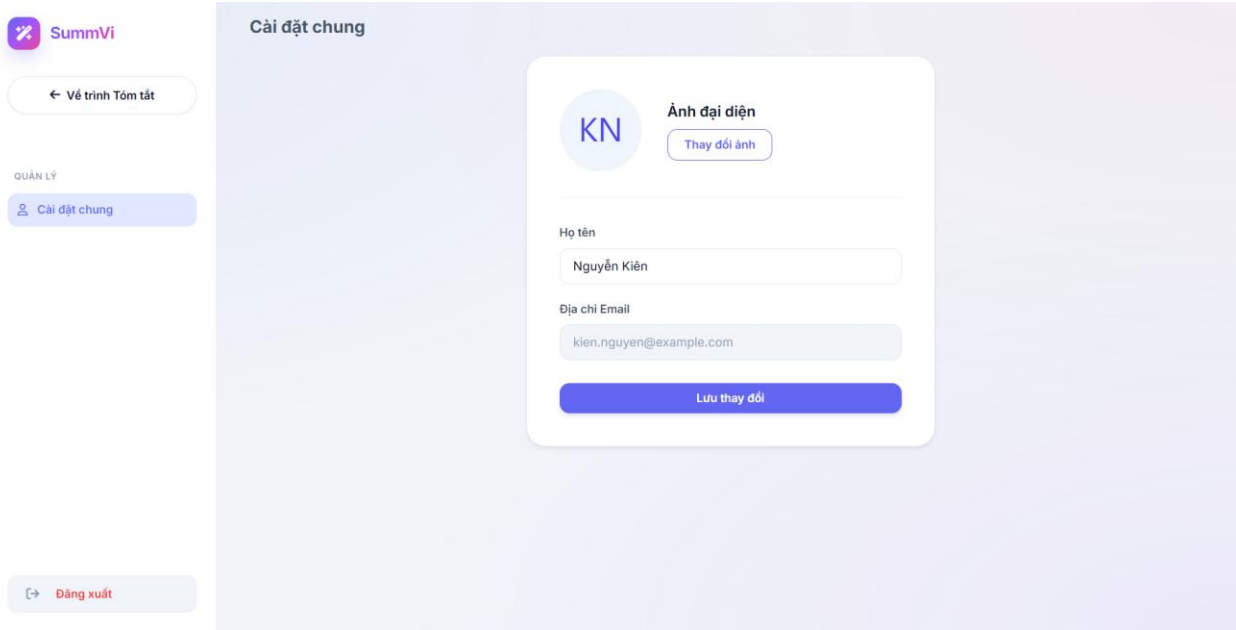
Ảnh: Giao diện chính

3.4.4 Giao diện hiển thị kết quả tóm tắt



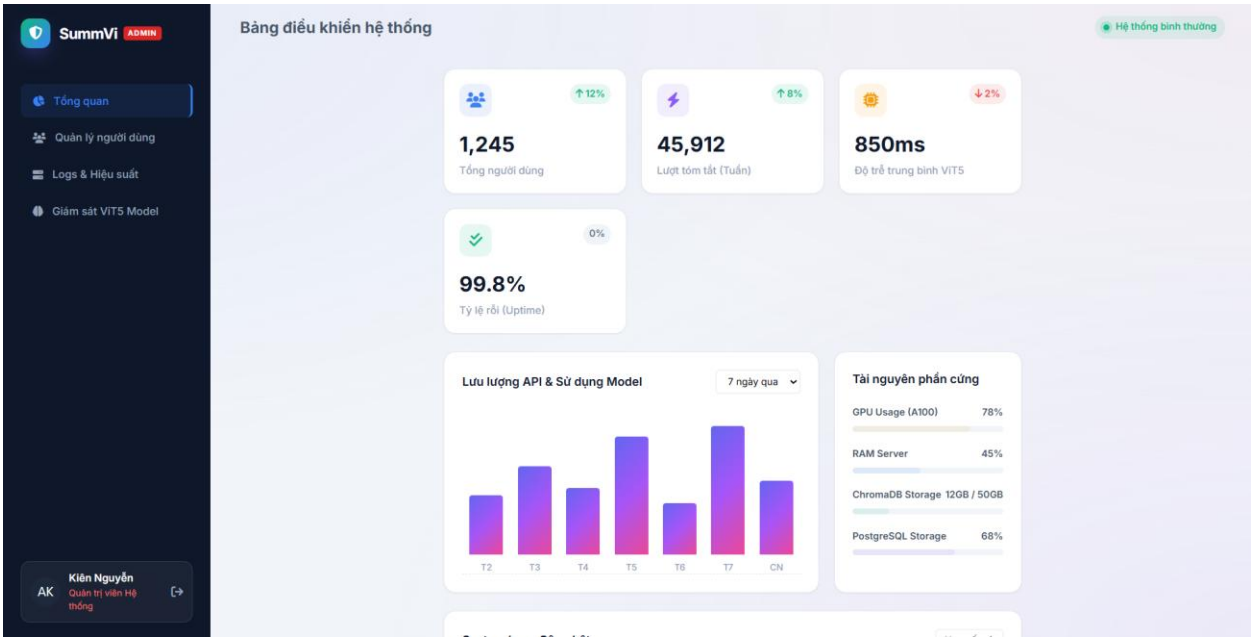
Ảnh: Giao diện hiển thị kết quả tóm tắt

3.4.5 Giao diện cài đặt tài khoản



Ảnh: Giao diện cài đặt tài khoản

3.4.6 Giao diện quản lý hệ thống (dành cho quản trị viên)



Ảnh: Giao diện quản lý hệ thống